DEBUGGING

DEBUGGEL -> DEBUGGER

LEACT -> REACT

TRÁPNY VTIP -> 😬

VIETE UŽ DEBUGGOVAŤ?

DEBUGGING

DEBUGGING

Metodický proces identifikácie, analýzy a odstraňovania chýb (tzv. bugov, porúch alebo nedostatkov) zo softvéru alebo hardvéru.

KĽÚČOVÉ ASPEKTY

- Proces
- Chyby
- Techniky
- Nástroje

STABILITA

A

BEZPEČNOSŤ


TRANSPARENTNÝ KÓD
EFEKTÍVNA SPRÁVA
MENEJ CHÝB VIAC RIEŠENÍ
Debugging
VIAC LOGIKY MENEJ CHAOSU
Stabilita a bezpečnosť
Konečne update, ktorý dáva zmysel.
</>
MENEJ CHÝB V POLITIKE
VIAC BEZPEČNOSTI
LEPŠIE BUDÚCNOSTI
Debugging.sk

NÁSTROJE

STATICKÁ ANALÝZA KÓDU

Čo si myslíte, že to znamená?

STATICKÁ ANALÝZA KÓDU

Statická analýza je kontrola zdrojového kódu bez jeho spustenia.

TYPESCRIPT

Je nadstavba programovacieho jazyka JavaScript.
Umožňuje vývojárom definovať typy údajov.

DEMO

ESLINT

Séria individuálne konfigurovateľných pravidiel, ktoré sa navzájom neovplyvňujú.

DEMO

IDE

INTEGRATED DEVELOPMENT ENVIRONMENT

Je aplikácia, ktorá spája všetky nástroje potrebné na programovanie do jedného rozhrania.

AKÉ IDE POZNÁTE/POUŽÍVATE?

REACT NATIVE DEVTOOLS

Hlavný nástroj na debugging React Native aplikácie od verzie 0.76.

CONSOLE

Miesto, kde môžeme čítať klasické

- logy -> console.log
- varovania -> console.warn
- chyby -> console.error

LOGBOX ERRORS AND WARNINGS

LogBox je systém na zobrazovanie chýb a varovaní. Jeho hlavným cieľom je urobiť debugging aplikácií menej frustrujúcim a prehľadnejším.

DEMO

NÁJDITE CHYBU VO VIDEU Č. 1



NÁJDITE CHYBU VO VIDEU Č. 2



NÁJDITE CHYBU VO VIDEU Č. 3



SOURCES, DEBUGGER, BREAKPOINTS

Sources je miesto, kde analyzujeme logiku, sledujeme kód a hľadáme chyby v algoritmoch. Je to 'riadiace centrum' procesu ladenia.

DEMO

NETWORK

Network zobrazuje sieťovú komunikáciu medzi JavaScriptovou časťou aplikácie a serverom (API). Požiadavky (requesty) odoslané natívnym kódom aplikácie, operačného systému alebo iných aplikácií sa v tejto sekcii nezobrazujú.

DEMO

PERFORMANCE

Performance je pokročilý diagnostický nástroj určený na hĺbkovú analýzu správania aplikácie v reálnom čase. Poskytuje komplexný pohľad na interakciu medzi JavaScriptovým vláknom (Hermes engine), plánovačom úloh (Scheduler) a vykresľovacím cyklom.

DEMO

MEMORY

Memory je nástroj na analýzu spotreby pamäte JavaScriptovej časti aplikácie. Pomáha odhaliť **memory leaky**

GARBAGE COLLECTOR

JavaScriptové interpretery (engines) implementujú garbage collector (GC), ktorý automaticky vyhľadáva nepotrebné objekty v pamäti a uvoľňuje priestor na ďalšie použitie.

MEMORY LEAK

Memory leak je situácia, kedy aplikácia postupne zaberá čoraz viac pamäte a neuvoľňuje ju. Výsledkom môže byť spomalenie alebo pád aplikácie. Memory

PRÍKLADY KÓDU, KTORÝ SPÔSOBUJE MEMORY LEAK

LISTENER, KTORÝ NEPRESTANE POČÚVAŤ

```
1 const BadEventComponent = () => {
2   useEffect(() => {
3     const subscription = EventEmitter.addListener('myEvent', handleEvent);
4     // Chýba cleanup funkcia – subscription sa nikdy nezruší
5   }, []);
6   return <text>Listening for events...</text>;
7 };
```

TIMER, KTORÝ NEPRESTANE BEŽAŤ

```
const BadTimerComponent = () => {
  const [count, setCount] = useState(0);
  useEffect(() => {
    const timer = setInterval(() => {
      setCount(prev => prev + 1);
    }, 1000);
    // Chýba cleanup funkcia – timer sa nikdy nevymaže
  }, []);
  return <text>Count: {count}</text>;
};
```

CLOSURE, KTORÁ DRŽÍ REFERENCIU NA VEĽKÝ OBJEKT

```
// ❌ Problém
class BadClosureExample {
  private largeData: string[] = new Array(1000000).fill('some data');

  createLeakyFunction(): () => number {
    // Celé pole largeData je zachytené v closure
    return () => this.largeData.length;
  }
}

// ✅ Riešenie
class GoodClosureExample {
  private largeData: string[] = new Array(1000000).fill('some data');

  createEfficientFunction(): () => number {
    // Do closure sa zachytí len samotná dĺžka, nie celé pole
    const length = this.largeData.length;
    return () => length;
  }
}
```

DEMO

COMPONENTS

Components je nástroj na vizuálnu inšpekciu stromu React komponentov v reálnom čase. Umožňuje preskúmať štruktúru aplikácie, zobraziť aktuálny stav (state) a vlastnosti (props) každého komponentu.

DEMO

PROFILER

Je nástroj na meranie výkonu React komponentov z pohľadu renderovania. Nahrá, ako dlho trvalo vykresliť každý komponent počas interakcie s aplikáciou, a pomáha identifikovať, ktoré komponenty sa renderujú zbytočne alebo príliš pomaly.

DEMO

AKO DEBUGOVAŤ CHYBU – POSTUP

1. Prečítať chybovú správu
2. Vyhľadať chybu (Google / AI)
3. Izolovať problematický kód
   - Nájdite commit, po ktorom sa chyba objavila.
   - Dočasne odstraňujte časti kódu.
   - Vytvorte minimálny reprodukčný príklad.
4. Použiť breakpoint pri chybe

PRODUKČNÉ CHYBY

Chyby, ktoré sa objavia až u reálnych používateľov, sú najťažšie na riešenie, pretože k nim nemáme priamy prístup.

1. Reprodukovať lokálne
2. Chyby špecifické pre zariadenie
3. Sledovanie pomocou reporting systému

REPORTING SYSTEM – CRASHLYTICS

Je najpoužívanejší reporting nástroj pre mobilné aplikácie. Po integrácii do aplikácie automaticky zachytáva všetky nekritické aj fatálne chyby.

CVIČENIE

Upravte súbor PracticeDebuggingScreen.tsx čo najefektívnejšie.